



Etapas en una agregación

Una de las características más poderosas de MongoDB es su capacidad de realizar **agregaciones**, las cuales permiten transformar, filtrar y resumir conjuntos de datos complejos mediante una estructura de procesamiento por etapas. A continuación, aprenderemos cómo funcionan estas etapas, cómo implementarlas en la herramienta Compass y cómo utilizarlas para obtener distintos tipos de análisis a partir de un conjunto de datos reales.

En general, en MongoDB se pueden utilizar agregaciones para:

- Agrupar valores de múltiples documentos.
- Ejecutar operaciones en los datos agrupados para obtener un único resultado.
- Analizar cambios en la información sobre el tiempo.

En MongoDB, los operadores de agregación se aplican en una estructura tipo “tubería”, donde la salida de un operador se utiliza como entrada para el siguiente operador. En inglés, esta estructura se conoce como *pipeline* y es uno de los términos más usados, incluso en español.

Este *pipeline* consiste en una o más etapas que procesa documentos:

- Cada etapa dentro del *pipeline* **agrupa, modifica o agrega** los datos contenidos en los documentos y luego envía el resultado al siguiente operador. Por ejemplo, una etapa puede filtrar documentos, agrupar documentos o calcular valores.
- Los documentos resultantes de una etapa pasarán a la siguiente etapa.
- El *pipeline* puede regresar resultados para grupos de documentos, por ejemplo, regresar el total, el promedio, el mínimo y el máximo.

Observa la siguiente figura:



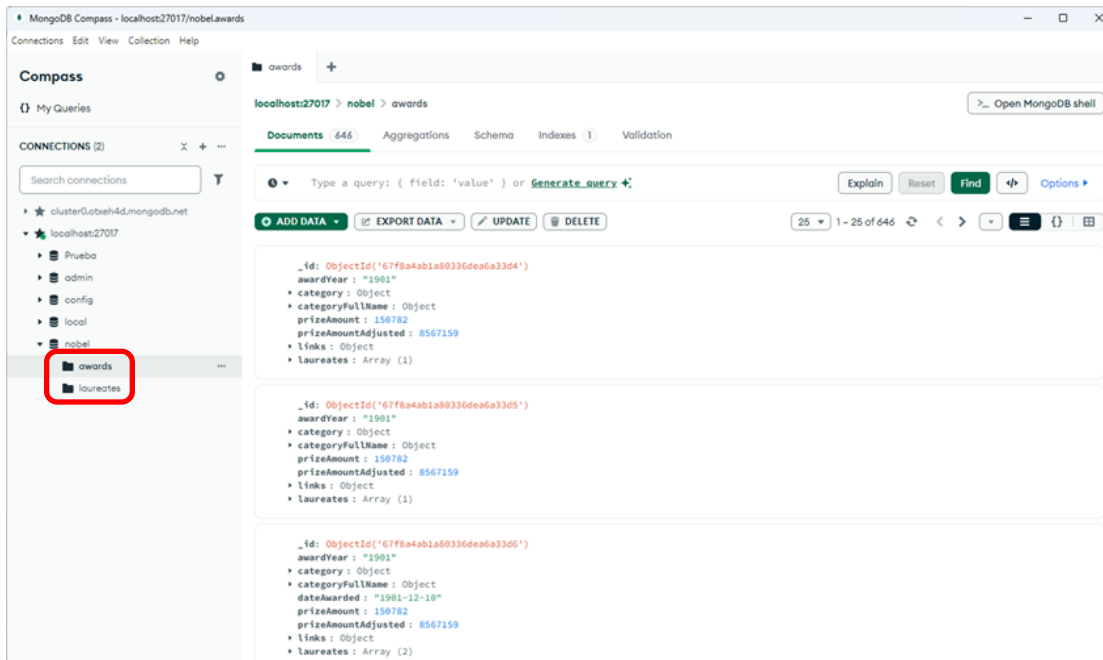
Como el resto de las estructuras en MongoDB, las agregaciones se forman por **documentos BSON** con un formato específico, donde el *pipeline* se representa como un arreglo y cada etapa como un objeto BSON. Por ejemplo, un *pipeline* o agregación con tres etapas, se representaría de la siguiente manera:

```
[etapa1, etapa2, etapa3]
```

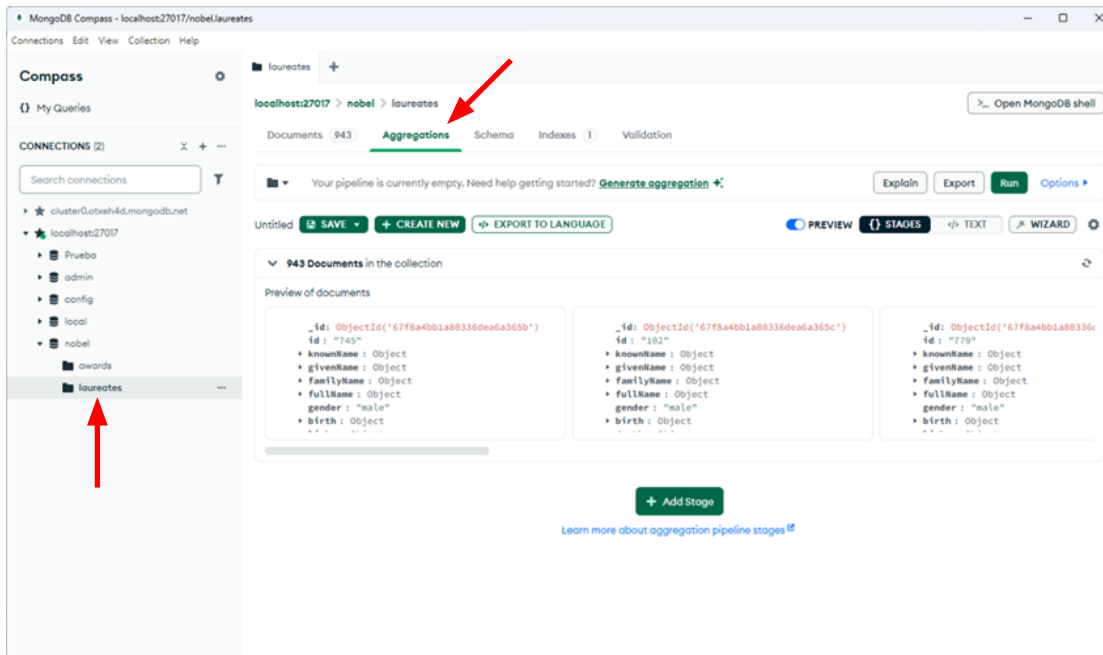
Donde *etapa1*, *etapa2* y *etapa3* son documentos bien formados de MongoDB con la sintaxis correcta de agregaciones.

Para aprender a hacer agregaciones, utilizaremos el programa Compass, que aprendiste a usar en las lecciones anteriores, así como el conjunto de datos *Nobel Prize Dataset* de Kaggle. Encontrarás la liga para acceder a este sitio en la sección de Material adicional, al terminar esta lectura. Este conjunto de datos incluye dos colecciones: `json_award.json` y `json_laureates.json`. **Asegúrate de importar ambos archivos en colecciones separadas.**

Por ejemplo, puedes crear una base de datos llamada `nobel`, con dos colecciones: `awards` y `laureates`.



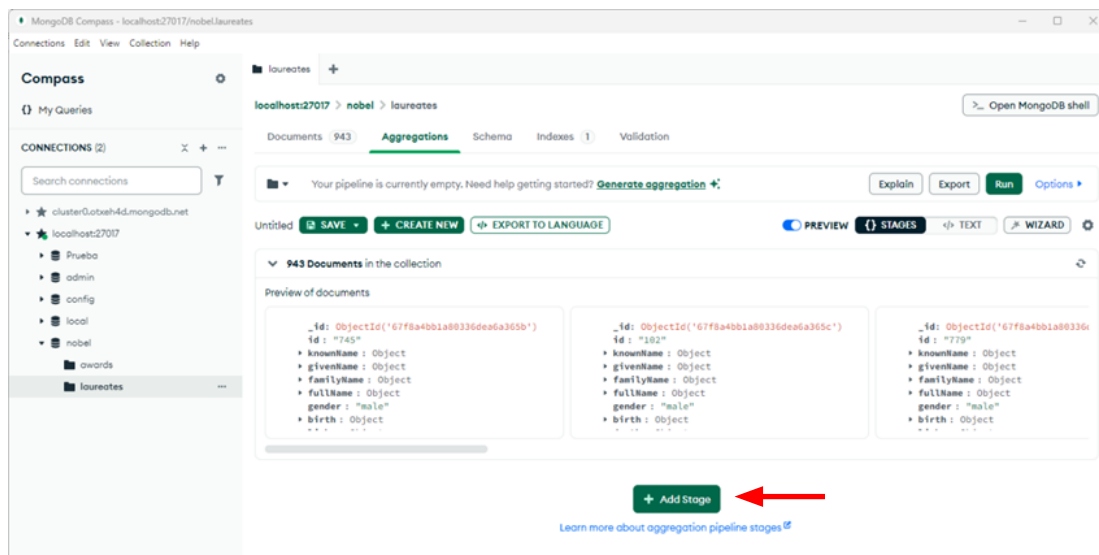
Vamos a comenzar con un ejemplo. En Compass, selecciona la colección que contiene los datos `laureates` y elige la opción **“Aggregations”**.



Por omisión, Compass desplegará una muestra del total de documentos y una primera etapa vacía.

El primer tipo de etapa que utilizaremos es de tipo `$match`, que filtra los documentos que cumplen ciertas condiciones y los envía a la siguiente etapa. Los documentos se envían sin ninguna modificación. La etapa `$match` utiliza operadores estándar de consultas de MongoDB.

Después, oprimimos el botón **“Add Stage”** para agregar una nueva etapa.



Por ejemplo, para el conjunto de datos que tenemos, si queremos filtrar y enviar a la siguiente etapa los documentos cuyo campo `gender` es `“female”`, utilizaremos la siguiente consulta dentro de la etapa:

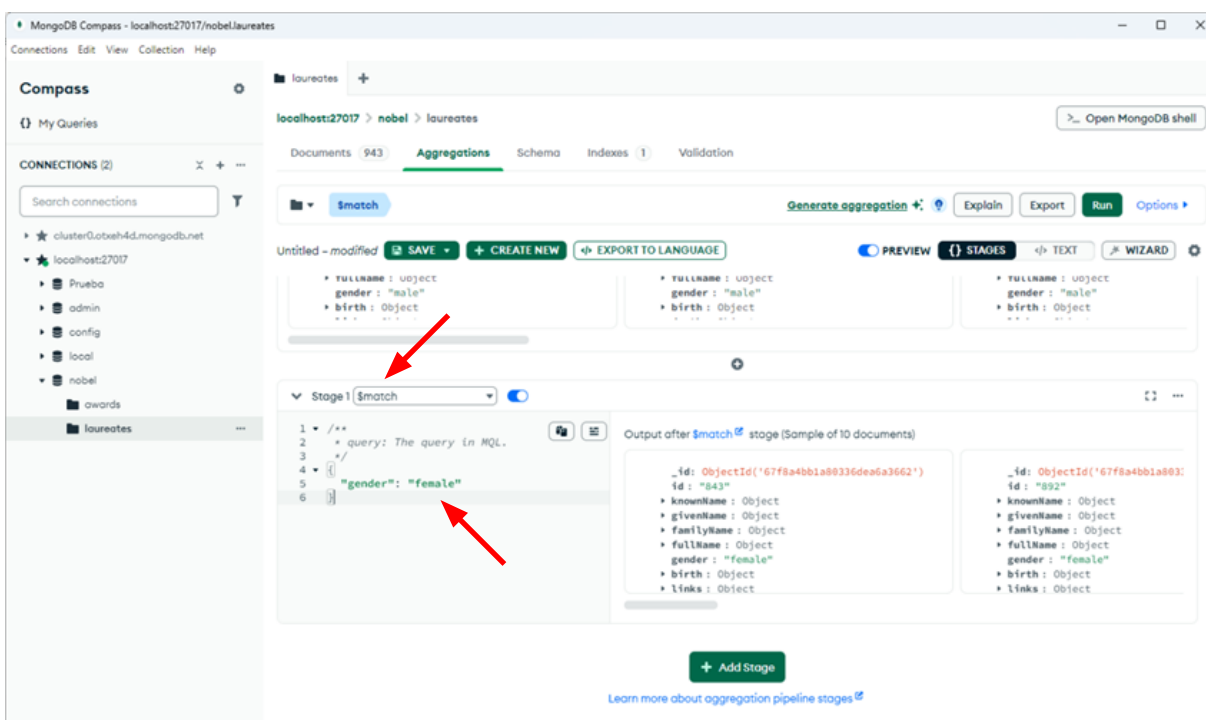
```
{
  "$match": {
    "gender": "female"
  }
}
```

Aunque la sintaxis completa de la etapa es como se muestra, al momento de seleccionar el tipo de etapa, Compass automáticamente agrega la base del documento y solo requiere el contenido de la etapa (marcado en rojo).

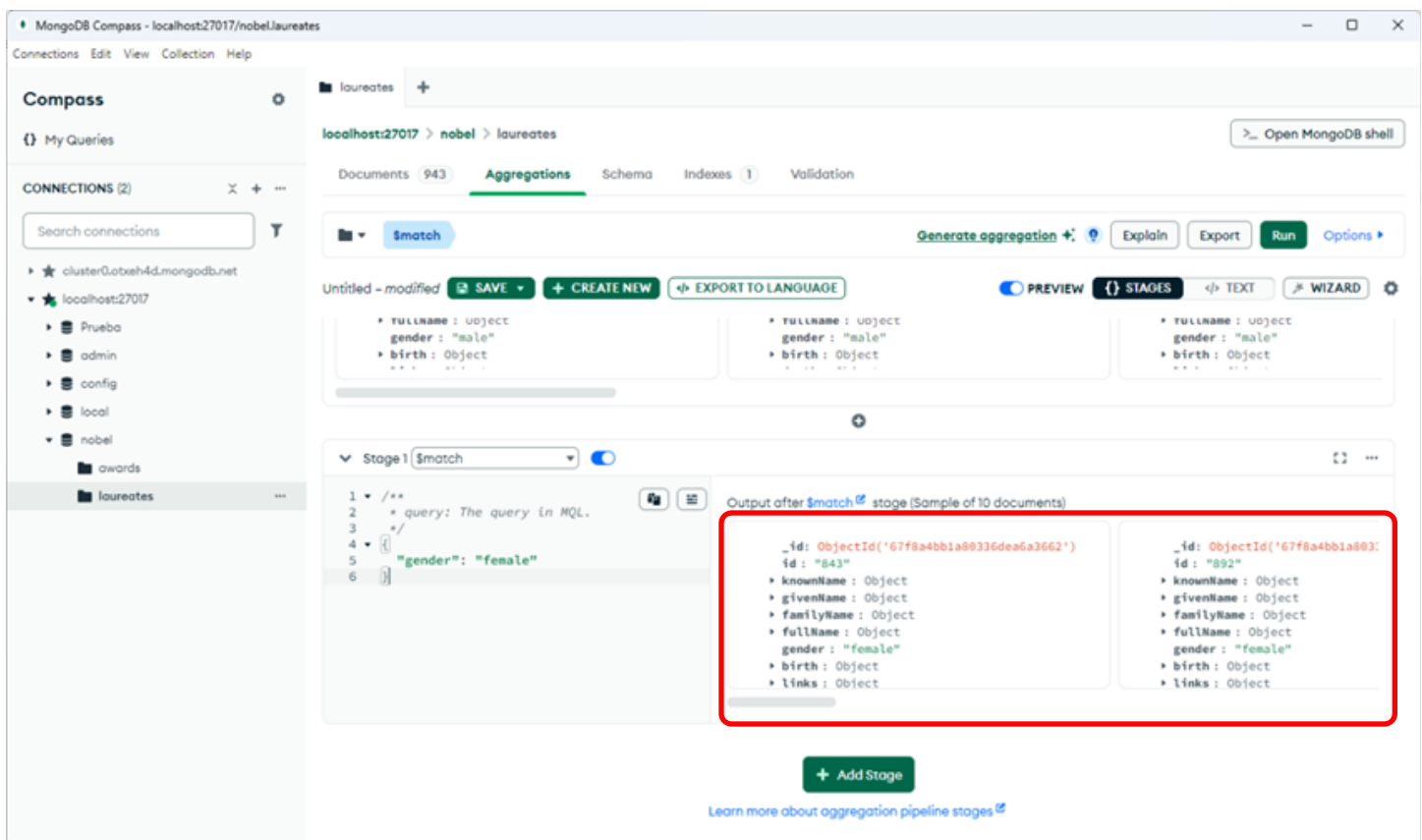
Además, Compass agrega texto con el siguiente formato:

```
/**
 * Texto
 */
```

Este texto es un comentario sobre el documento y **no** se considera al momento de ejecutar la etapa. Se puede ignorar o eliminar.



Es importante destacar que Compass solo desplegará una muestra de 20 resultados del lado derecho de la etapa, que no necesariamente representa el total de documentos que cumple con la condición del `$match`. El tamaño de esta muestra es configurable.



Para contar el total de documentos que cumplieron la condición, agregaremos una etapa `$count` oprimiendo de nuevo el botón **"Add Stage"**.

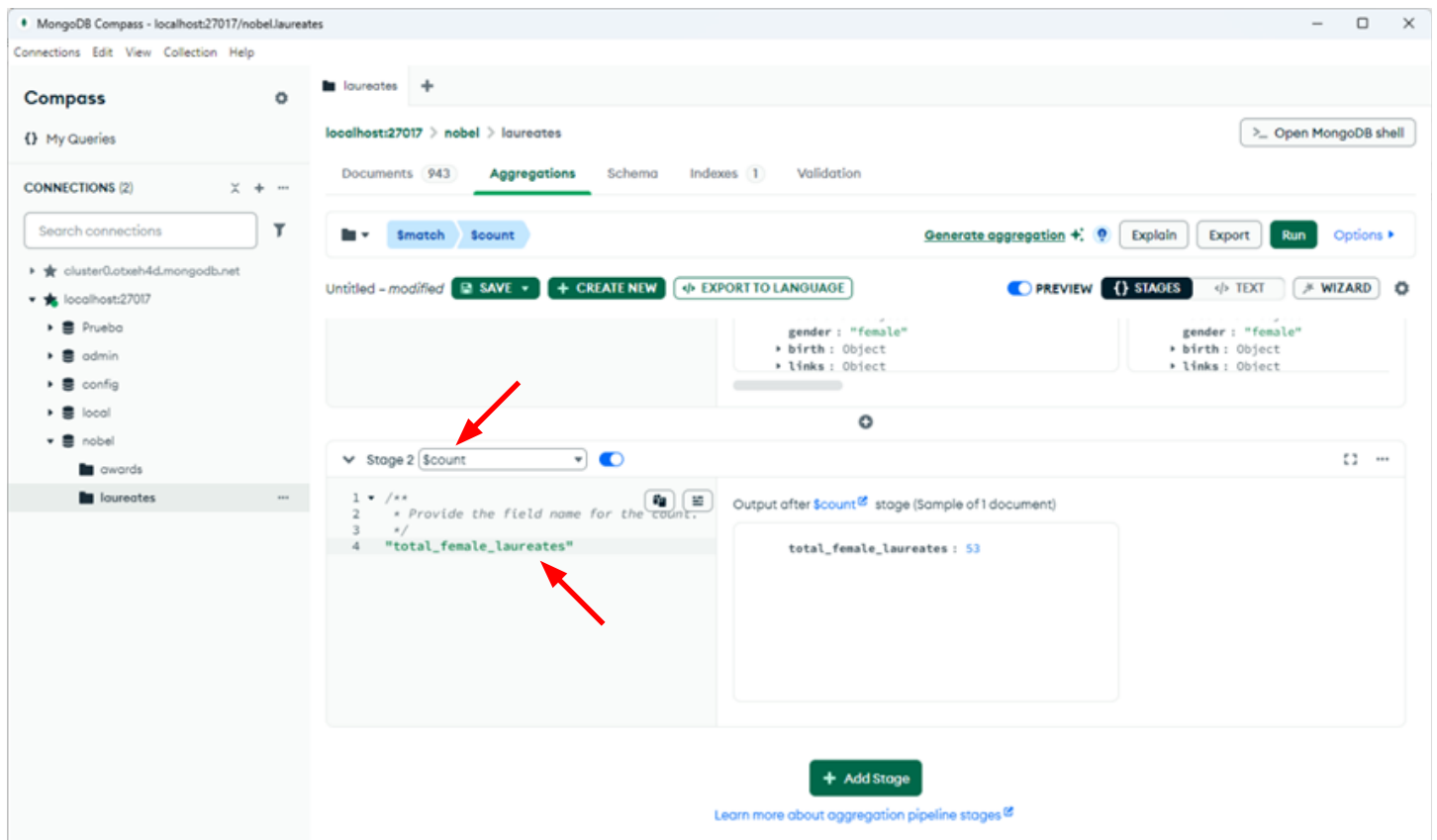
La sintaxis de este tipo de etapa es más simple, solamente hay que asignar un nombre al campo que contendrá el valor. Este nombre debe cumplir con las características de un campo de un documento de MongoDB. Por ejemplo,

podemos almacenar el resultado del conteo en un campo llamado `total_female_laureates`, como se muestra a continuación:

```
{
  "$count": "total_female_laureates"
}
```

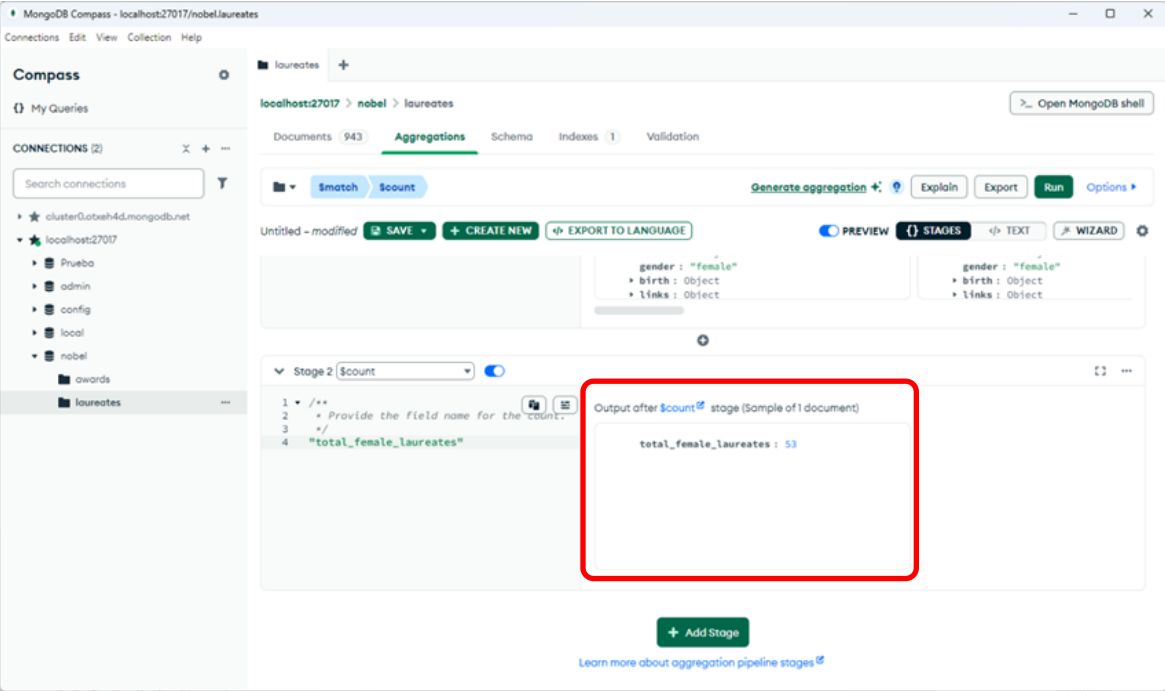
Al igual que con `$match`, Compass solo requiere la parte marcada en rojo y agrega de manera automática el resto del documento.

Durante el resto de la lección, indicaremos en **rojo** la parte del documento que se captura en Compass.



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017' with a database named 'nobel'. The 'Collections' list on the left includes 'laureates'. The main panel shows the 'Aggregations' tab for the 'laureates' collection. The aggregation pipeline has two stages: '\$match' and '\$count'. The '\$count' stage is selected, and its configuration is shown in the 'STAGES' panel. The field name for the count is 'total_female_laureates', which is highlighted in red in the original image. The 'Output after \$count stage (Sample of 1 document)' shows a document with the field 'total_female_laureates' and the value 53.

Observa cómo \$count solo regresa un documento:



El resultado de la agregación nos indica que ha habido 53 mujeres ganadoras del premio Nobel. Analizando los campos de los documentos, ¿se te ocurre alguna otra agregación con estos dos tipos de etapa?

MongoDB ofrece distintos tipos de etapas, algunos de los más comunes son:

Etapa	Descripción
\$addFields	Agrega un campo nuevo a cada documento que pasa por la etapa. \$set es otro nombre para \$addFields.
\$count	Cuenta el número de documentos en esta etapa del <i>pipeline</i> .
\$group	Agrupa los documentos por un identificador y, si está definida, aplica una expresión de acumulación a cada grupo. Regresa un documento por cada grupo distinto. Los documentos de salida solamente contienen el campo identificador y, si se especificaron, los campos de acumulación.

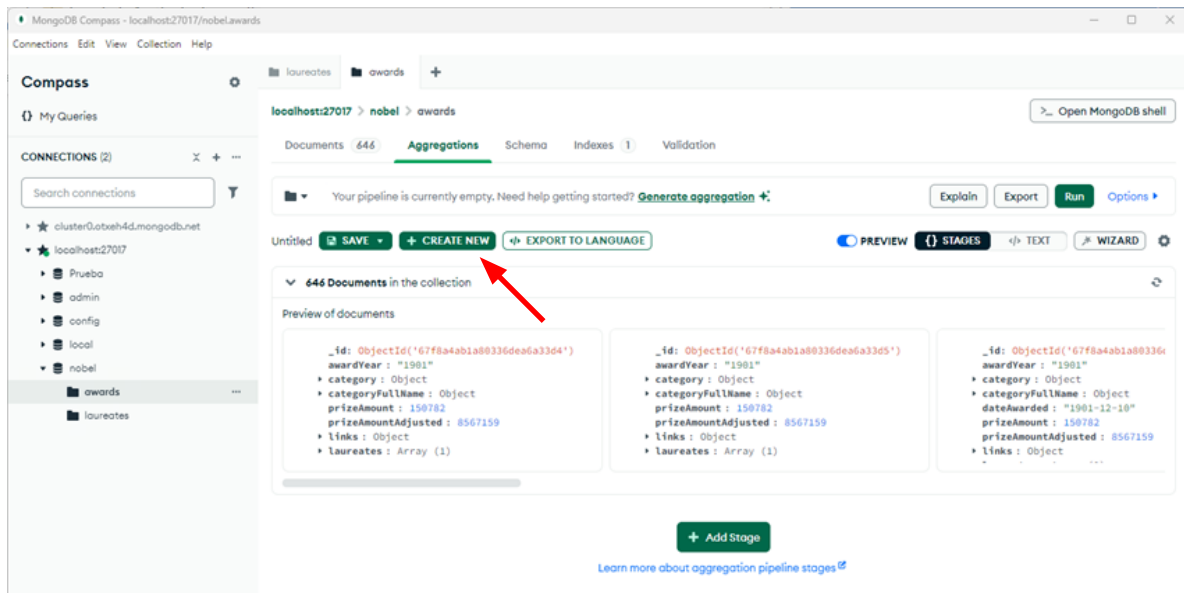
Etapa	Descripción
\$limit	Pasa a la siguiente etapa los primeros <i>n</i> documentos, donde <i>n</i> es un número.
\$match	Filtra los documentos y solo envía a la siguiente etapa aquellos que cumplan con la condición.
\$merge	Escribe todos los documentos recibidos en una colección especificada. Agrega los documentos a la colección si ya existe, y puede sobrescribir documentos. Debe ser la última etapa en el <i>pipeline</i> .
\$out	Escribe todos los documentos recibidos en una colección especificada. Reemplaza la colección si ya existe. Debe ser la última etapa en el <i>pipeline</i> .
\$project	Pasa los documentos con los campos especificados a la siguiente etapa. Los campos especificados pueden ser existentes o nuevos campos calculados.
\$sort	Ordena los documentos por un campo específico.

Puedes encontrar la lista completa de etapas en la documentación oficial de MongoDB. Encontrarás la liga para acceder a este sitio en la sección de Bibliografía.

Ejemplos

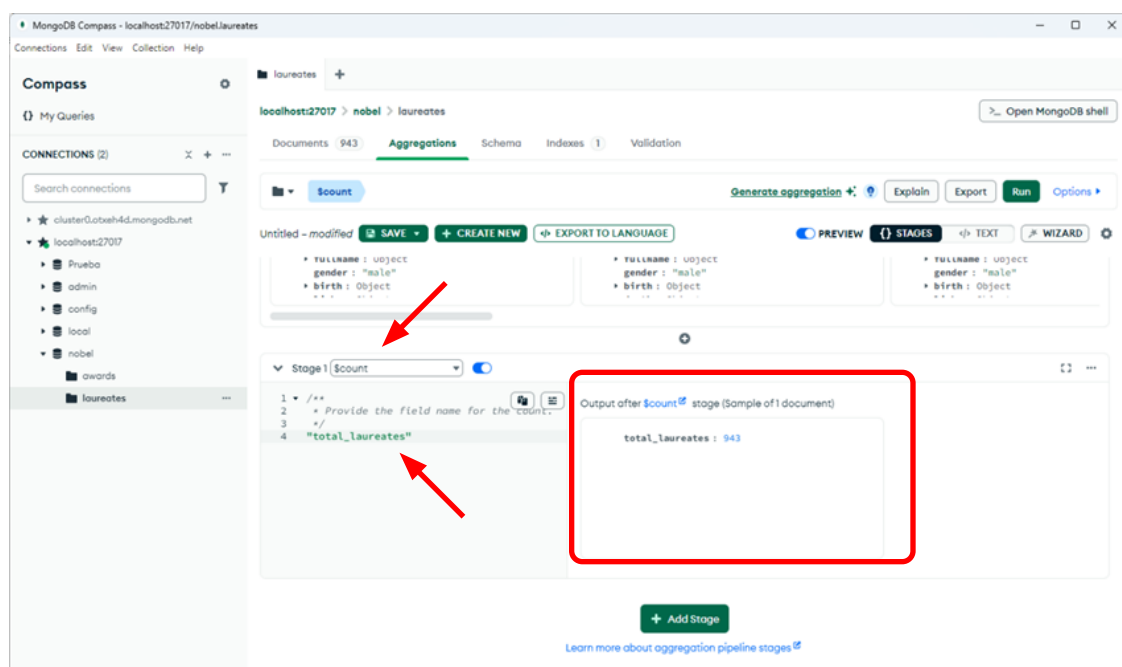
A continuación, vamos a hacer ejemplos utilizando los tipos de etapa listados. Para cada ejemplo, expresaremos el resultado deseado y luego mostraremos los pasos de la agregación.

Como primer ejemplo, vamos a contar el total de premios entregados en una nueva agregación. Para crearla, simplemente seleccionamos el botón **“Create New”**.



Esta es una agregación muy simple, ya que consiste solamente de una etapa \$count:

```
{
  "$count": "total_laureates"
}
```



Podemos ver que, para este conjunto de datos, el total de documentos es de 943, lo cual indicaría el número de Premios Nobel entregados.

Continuando con otro ejemplo, al analizar los datos de la colección `laureates`, podemos observar que existe el campo `birth`, que consiste en un **objeto** que contiene la fecha de nacimiento en formato de cadena de caracteres, y otro objeto anidado llamado `place`, que a su vez contiene la ciudad, el país, y otros campos en varios idiomas. Para nuestros ejercicios utilizaremos los campos nombrados `en`, que es el nombre en inglés de cada ubicación.

Además, podemos notar que algunos documentos contienen el campo `death`, con una estructura similar a `birth`. Podemos asumir que, si un documento no tiene el campo `death`, indica que la persona premiada aún vive.

Con esta información, queremos saber el número de personas premiadas que aún vivían hasta el momento en que el conjunto de datos fue recolectado.

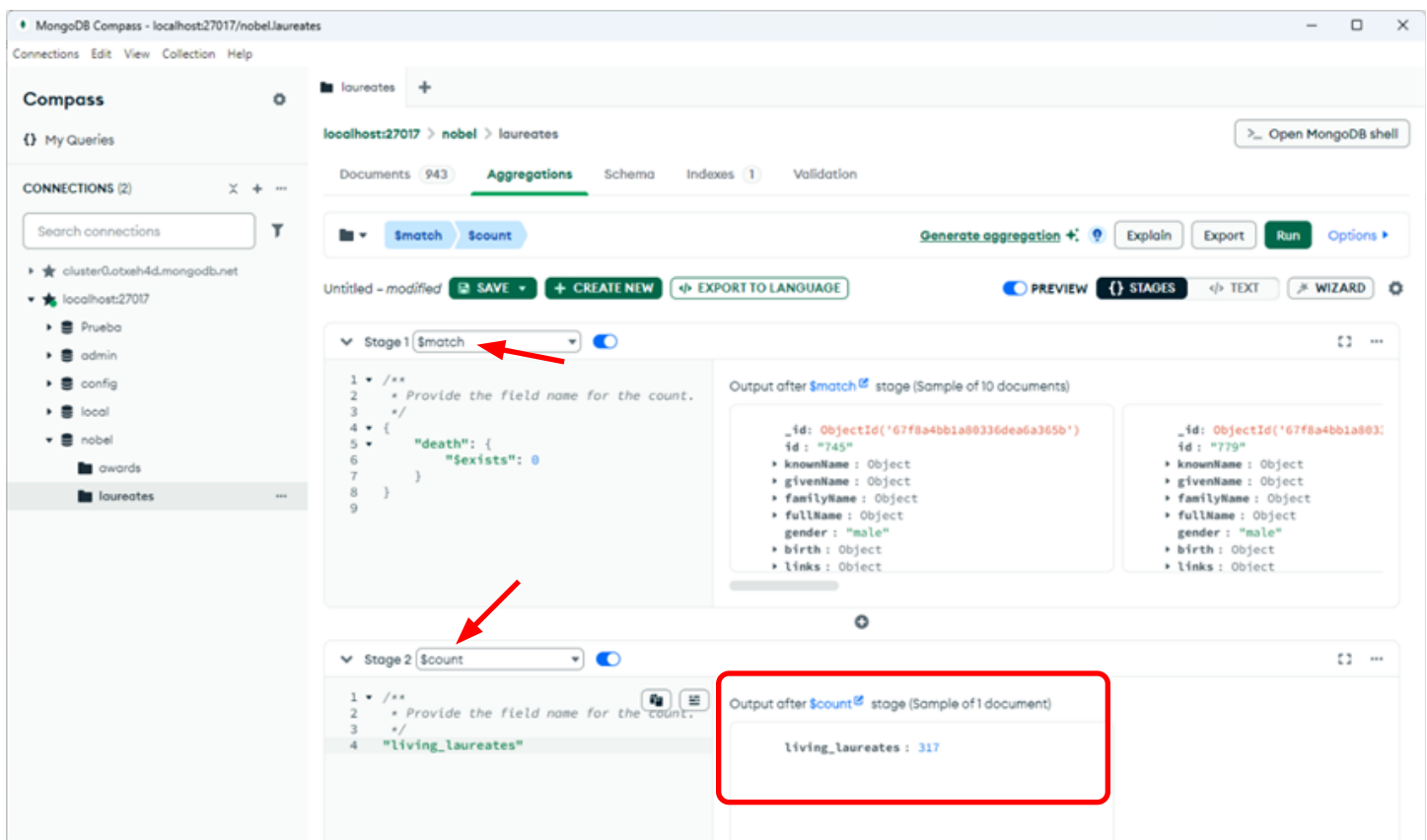
Una manera de obtener este dato es filtrando los documentos que **no** contengan el campo `death` y contando el número de documentos. Podemos lograr esto con una nueva agregación de dos etapas:

1. Una primera etapa tipo `$match` para filtrar los documentos que **no** contengan el campo `death`:

```
{
  "$match":
  {
    "death": {
      "$exists": 0
    }
  }
}
```

2. Una etapa `$count` para contar los documentos resultantes. Al campo donde se almacenará el resultado le llamaremos `living_laureates`:

```
{
  "$count": "living_laureates"
}
```

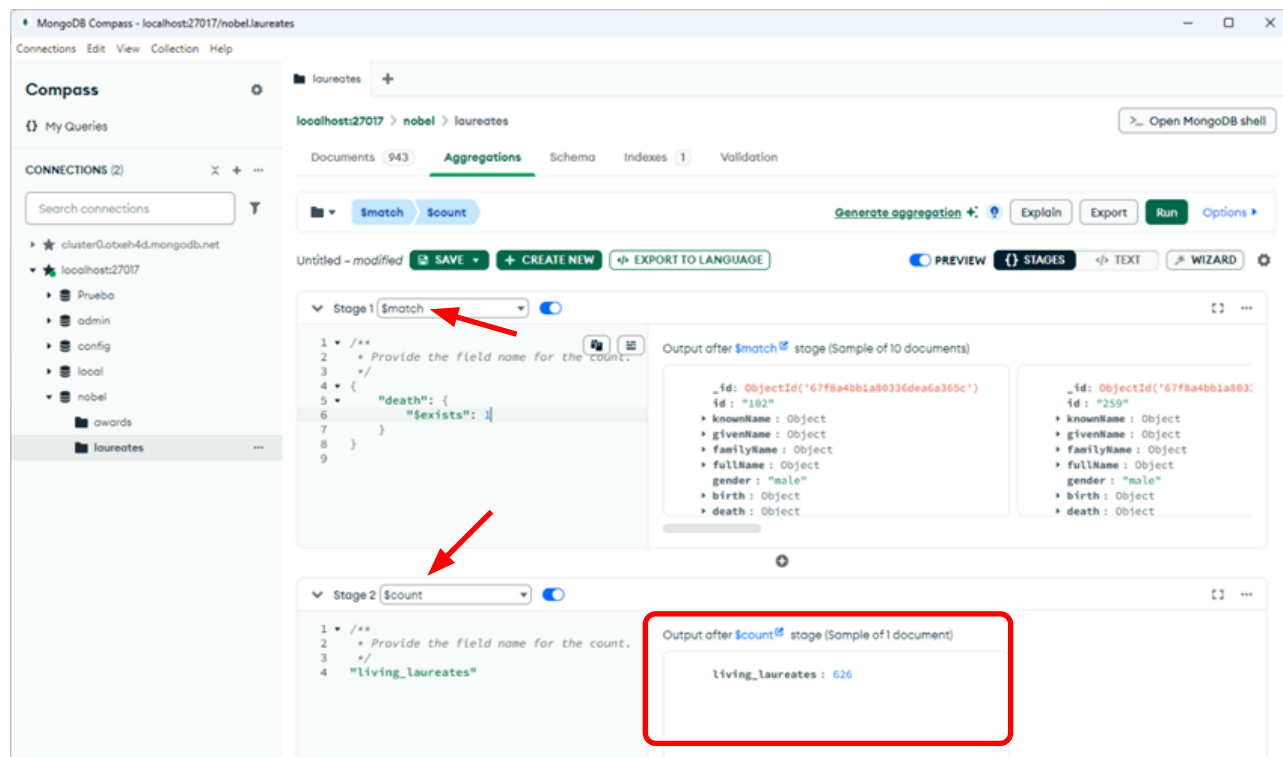


Podemos observar que tenemos 317 premiados vivos.

Por otro lado, si queremos saber el número de premiados fallecidos, simplemente podemos cambiar la etapa `$match` a:

```
{
  "$match": {
    "death": {
      "$exists": 1
    }
  }
}
```

Y obtendremos que el número de premiados fallecidos es de 626.



Las agregaciones nos permiten **agrupar documentos por un campo específico**. Por ejemplo, si quisiéramos saber el número de personas premiadas por país de origen, podríamos hacer un *pipeline* con una etapa `$match` con el nombre del país y luego una etapa `$count`. Sin embargo, si tenemos muchos valores para el país, es inviable hacer tantos *pipelines* diferentes.

Así que, para este caso, podemos utilizar la etapa `$group`, que permite agrupar por un campo o lista de campos. La sintaxis de la etapa `$group` es la siguiente:

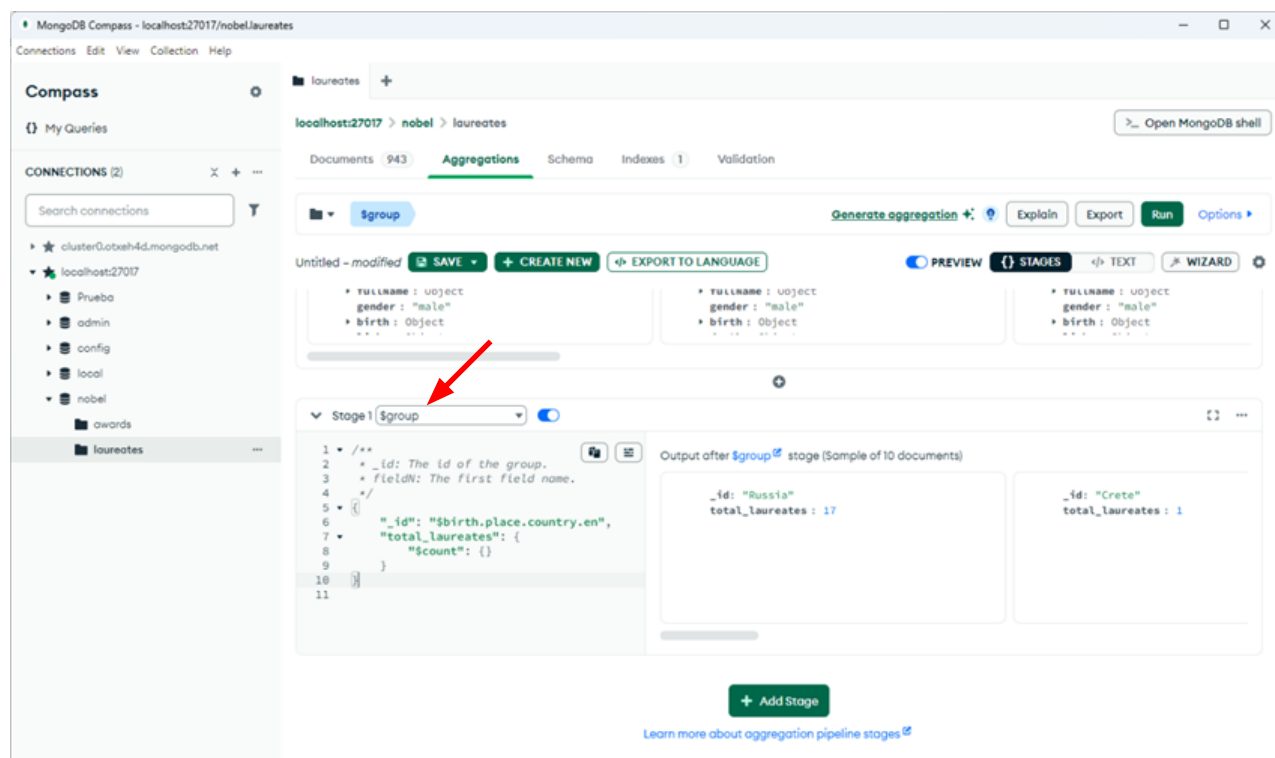
```
{
  $group:
  {
    _id: <expression>,
    <field1>: {<accumulator1>: <expression1>},
    ...
    <fieldN>: {<accumulatorN>: <expressionN>},
  }
}
```

Donde en `<expression>` se indicará el nombre del campo o campos por el cual se agrupará. Los campos `<field1>` a `<fieldN>` son opcionales e indican la aplicación de operadores de acumulación sobre los datos agrupados.

Importante: Cuando el nombre de un campo aparece del lado derecho de los dos puntos, deberá incluir el prefijo `$`. Esto indica que se quiere referenciar el valor del campo.

Regresando al ejemplo, para agrupar por país y contar el número de documentos por cada grupo, podemos usar la siguiente etapa `$group`:

```
{
  "$group":
  {
    "_id": "$birth.place.country.en",
    "total_laureates": {
      "$count": {}
    }
  }
}
```



Aquí estamos utilizando el operador de agregación `$count` (que **no** hay que confundir con la etapa de tipo `$count`). Los **operadores de agregación** se utilizan dentro de las etapas para manipular, acumular, filtrar, concatenar, etcétera. Algunos de los operadores de agregación más comunes son:

Operador	Tipo	Descripción
<code>\$abs</code>	Aritmético	Regresa el valor absoluto de un número.
<code>\$add</code>	Aritmético	Regresa el resultado de la suma de los números. También puede sumar un número a una fecha y regresar una nueva fecha.
<code>\$divide</code>	Aritmético	Regresa el resultado de dividir el primer número entre el segundo.
<code>\$multiply</code>	Aritmético	Multiplica números y regresa el producto.

Operador	Tipo	Descripción
\$subtract	Aritmético	Regresa el resultado de restar el segundo valor del primero.
\$and	Booleano	Regresa <code>true</code> solamente si todas las expresiones son verdaderas.
\$not	Booleano	Regresa el valor booleano, que es el opuesto de la expresión parámetro.
\$or	Booleano	Regresa <code>true</code> si cualquiera de las expresiones es verdadera.
\$size	Arreglo	Regresa el número de elementos en un arreglo.
\$cond	Condicional	Un operador ternario que evalúa una expresión y, dependiendo del resultado, regresa una de las otras dos expresiones.
\$ifNull	Condicional	Si la primera expresión no es nula, regresa su valor; de lo contrario, regresa el valor de la segunda expresión.
\$concat	Cadena de caracteres	Concatena todas las expresiones en una cadena de caracteres.
\$toLower	Cadena de caracteres	Convierte una cadena de caracteres a minúsculas.
\$toUpper	Cadena de caracteres	Convierte una cadena de caracteres a mayúsculas.
\$isNumber	Tipo de datos	Regresa <code>true</code> si la expresión puede evaluarse como número.
\$avg	Acumulación	Regresa la media aritmética de los valores. Ignora los valores no numéricos.
\$count	Acumulación	Cuenta el número de documentos en un grupo.
\$first	Acumulación	Regresa el primer documento de un grupo.
\$last	Acumulación	Regresa el último documento de un grupo.

Operador	Tipo	Descripción
\$max	Acumulación	Regresa la expresión con mayor valor para cada grupo.
\$min	Acumulación	Regresa la expresión con mejor valor para cada grupo.
\$sum	Acumulación	Regresa una suma de valores numéricos. Ignora los valores no numéricos.

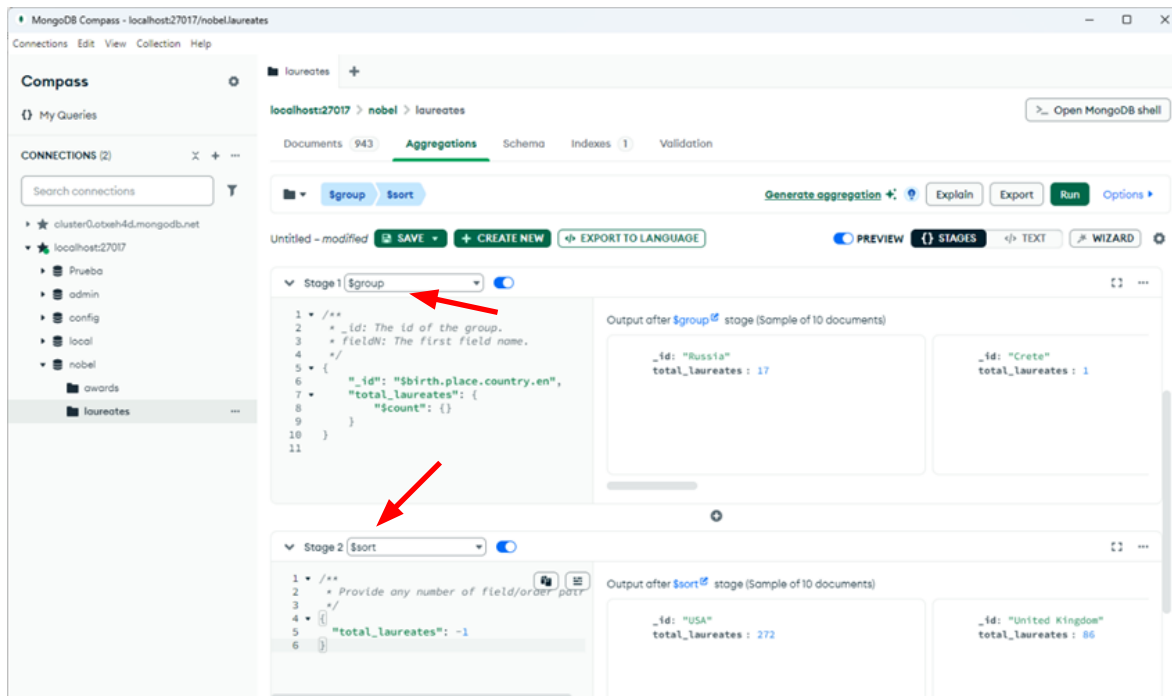
Es importante mencionar que **cada tipo de etapa define el tipo de operadores que puede recibir**. Por ejemplo, la etapa `$group` puede utilizar operadores de agregación de tipo Acumulación para definir nuevos campos.

Puedes consultar el listado completo de operadores de agregación y su sintaxis detallada en la documentación oficial de MongoDB. Encontrarás la liga para acceder a este sitio en la sección de Material adicional, al terminar esta lectura.

Regresando al ejemplo, después de agrupar por país y contar el número de documentos por grupo, podemos agregar una etapa `$sort` para ordenar los documentos resultantes de manera descendente por el número de premiados.

La sintaxis de la etapa `$sort` es sencilla, simplemente se indica el nombre del campo por el cual se ordenará y se le asigna el valor `1` para ordenamiento ascendente y `-1` para ordenamiento descendente:

```
{
  $sort:
  {
    "total_laureates": -1
  }
}
```



¿Recuerdas las agregaciones que utilizamos para determinar cuántos premiados aún viven y cuántos han fallecido? Con los operadores que acabamos de listar, podemos sacar el resultado en un solo *pipeline* con dos etapas:

Etapas 1:

```
{
  "$addFields": {
    "live": {
      "$cond": {
        "if": {
          "$ifNull": ["$death", false]
        },
        "then": "dead",
        "else": "live"
      }
    }
  }
}
```

Etapas 2:

```
{
  "$group":
  {
    "_id": "$live",
    "total_laureates": {
      "$sum": 1
    }
  }
}
```

¿Puedes interpretar qué está haciendo cada etapa?

The screenshot shows the MongoDB Compass interface for the 'laureates' collection. The aggregation pipeline is defined as follows:

```
1 //
2 * newField: The new field name.
3 * expression: The new field expression.
4 //
5 {
6   "live": {
7     "$cond": {
8       "if": {
9         "$ifNull": ["$death", false]
10      },
11       "then": "dead",
12       "else": "live"
13     }
14   }
15 }
```

Stage 1: \$addFields

Output after \$addFields stage (Sample of 10 documents):

- Document 1: `{ "_id": ObjectId('67f8a4bb1a80336dea6a365b'), "live": "dead", ... }`
- Document 2: `{ "_id": ObjectId('67f8a4bb1a80336dea6a365b'), "live": "live", ... }`

Stage 2: \$group

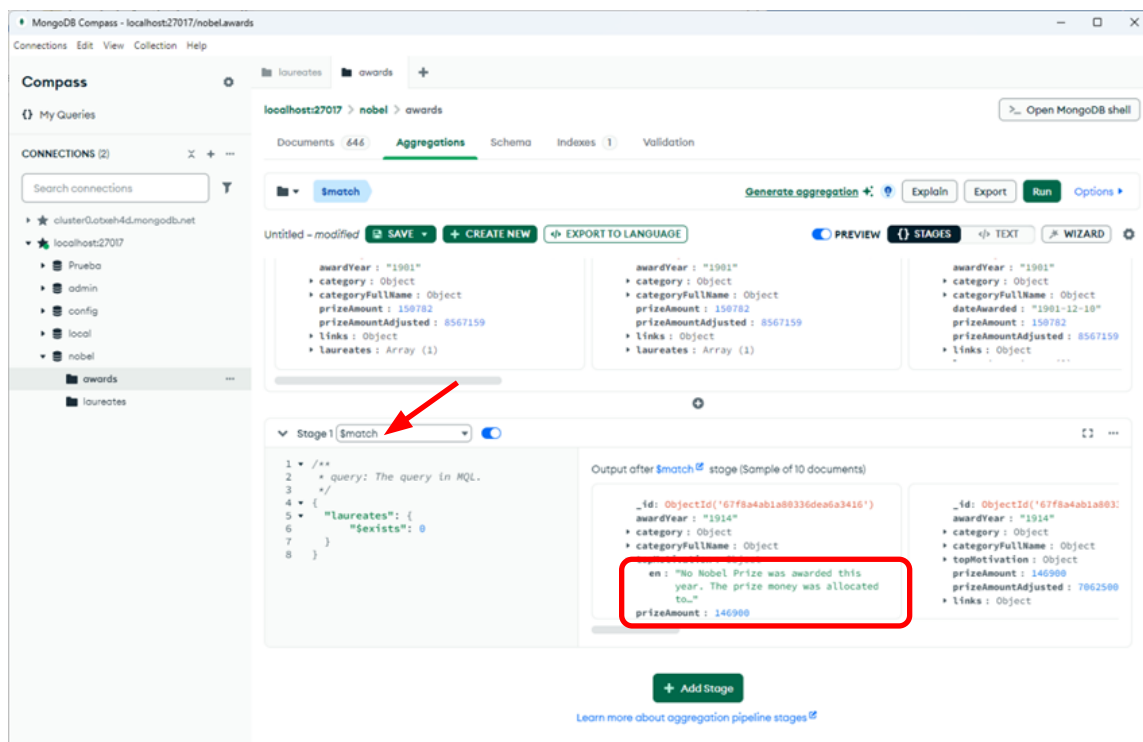
Output after \$group stage (Sample of 2 documents):

- Document 1: `{ "_id": "live", "total_laureates": 317 }`
- Document 2: `{ "_id": "dead", "total_laureates": 626 }`

Ahora, cambiemos a la colección `awards`. Analizando los documentos, podemos ver que hay algunos donde el campo `laureates` no existe, lo que indica que no hubo un premio asignado para la categoría y el año representados en el documento.

Reflexiona: ¿se podría generar un *pipeline* que liste todos los documentos representando el año y la categoría donde no hubo premiados?

```
{
  "$match": {
    "laureates": {
      "$exists": 0
    }
  }
}
```



Si solamente queremos saber el año y la motivación, podemos utilizar una etapa `$project`, que utiliza la sintaxis que vimos en la lección anterior:

```
{
  "$project":
  {
    "awardYear": 1,
    "topMotivation.en": 1,
    "_id": 0
  }
}
```

The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017'. The main panel displays the 'nobel' database with the 'awards' collection selected. The 'Aggregations' tab is active, showing a pipeline with two stages: '\$match' and '\$project'. The '\$match' stage is expanded, showing a query that filters for documents where 'sexists' is 0. The '\$project' stage is selected, and its configuration is shown in the 'Stage 2' dropdown. The configuration for the '\$project' stage is: `{ "awardYear": 1, "topMotivation.en": 1, "_id": 0 }`. The output of the '\$project' stage is displayed on the right, showing a sample of 10 documents. The output is a JSON object with the following fields: `awardYear`, `topMotivation`, and `links`. The `topMotivation` field is an object with an `en` property. A red arrow points to the '\$project' stage in the aggregation pipeline.

En otro ejemplo, utilizando la misma colección, queremos saber cuál fue el año en el que más personas u organizaciones recibieron el premio. Para lograrlo, podemos diseñar un *pipeline* de agregación con las siguientes etapas:

1. Filtrar únicamente aquellos documentos que tengan el campo `laureates`, lo cual indica que ese año sí se entregaron premios.
2. En los documentos resultantes, el campo `laureates` es un arreglo donde cada elemento representa a una persona u organización premiada. Podemos calcular cuántos premios se entregaron por año determinando la longitud de ese arreglo.
3. Ordenar los documentos resultantes en orden descendente según el número total de premiados.
4. Limitar los resultados a un solo documento para obtener únicamente el año con la mayor cantidad de premiaciones. Este documento final contendrá el dato que estamos buscando.

Las etapas para ejecutar el proceso anterior son:

Etapas 1:

```
{
  "$match":
  {
    "laureates": {
      "$exists": 1
    }
  }
}
```

Etapas 2:

```
{
  "$group":
  {
    "_id": "$awardYear",
    "total_laureates":
    {
      "$sum": {
        "$size": "$laureates"
      }
    }
  }
}
```

Etapas 3:

```
{
  "$sort":
  {
    "total_laureates": -1
  }
}
```

Etapas 4:

```
{
  "$limit": 1
}
```


MongoDB Compass - localhost:27017/nobel.awards

Connect View Collection Help

Local

DBS 7 COLLECTIONS

HOST localhost:27017

CLUSTER Standalone

EDITION MongoDB 5.0.6 Community

Filter your data

Prueba

admin

config

local

nobel

awards

laureates

nobel.awards

DOCUMENTS 646 STORAGE SIZE 221.2KB AVG. SIZE 1.1KB INDEXES 1 TOTAL SIZE 24.6KB AVG. SIZE 24.6KB

Documents Aggregations Schema Explain Plan Indexes Validation

COLLATION Untitled - Modified SAVE SAMPLE MODE AUTO PREVIEW

646 Documents in the Collection Preview of Documents in the Collection

\$match

Output after \$match stage (Sample of 20 documents)

```
1 {
2   "laureates": {
3     "sexists": 1
4   }
5 }
6 }
```

Output after \$match stage (Sample of 20 documents)

```
{
  "_id": ObjectId("623aa122106b28598a3cbe3"),
  "awardYear": "1981",
  "category": Object,
  "categoryFullName": Object,
  "prizeAmount": 158782,
  "prizeAmountAdjusted": 8567159,
  "links": Object,
  "laureates": Array
}
```

\$group

Output after \$group stage (Sample of 20 documents)

```
1 {
2   "_id": "$awardYear",
3   "total_laureates": {
4     "$sum": {
5       "$size": "$laureates"
6     }
7   }
8 }
9 }
10 }
```

Output after \$group stage (Sample of 20 documents)

```
{
  "_id": "1921",
  "total_laureates": 5
}
```

Output after \$group stage (Sample of 20 documents)

```
{
  "_id": "1937",
  "total_laureates": 7
}
```

> MONGOSH

MongoDB Compass - localhost:27017/nobel.awards

Connections Edit View Collection Help

Compass

My Queries

CONNECTIONS (2)

Search connections

cluster0-csh4d.mongodb.net

localhost:27017

Prueba

admin

config

local

nobel

awards

laureates

localhost:27017 > nobel > awards

Documents 646 Aggregations Schema Indexes 1 Validation

\$match \$group \$sort \$limit

Generate aggregation Explain Export Run Options

Untitled - modified SAVE CREATE NEW EXPORT TO LANGUAGE PREVIEW STAGES TEXT WIZARD

Stage 1 \$match

Output after \$match stage (Sample of 10 documents)

```
1 /**
2  * query: The query in MQL.
3  */
4 {
5   "laureates": {
6     "sexists": 1
7   }
8 }
```

Output after \$match stage (Sample of 10 documents)

```
{
  "_id": ObjectId("67f8a4ab1a80336dea33d4"),
  "awardYear": "1981",
  "category": Object,
  "categoryFullName": Object,
  "prizeAmount": 158782,
  "prizeAmountAdjusted": 8567159,
  "links": Object,
  "laureates": Array (1)
}
```

Stage 2 \$group

Output after \$group stage (Sample of 10 documents)

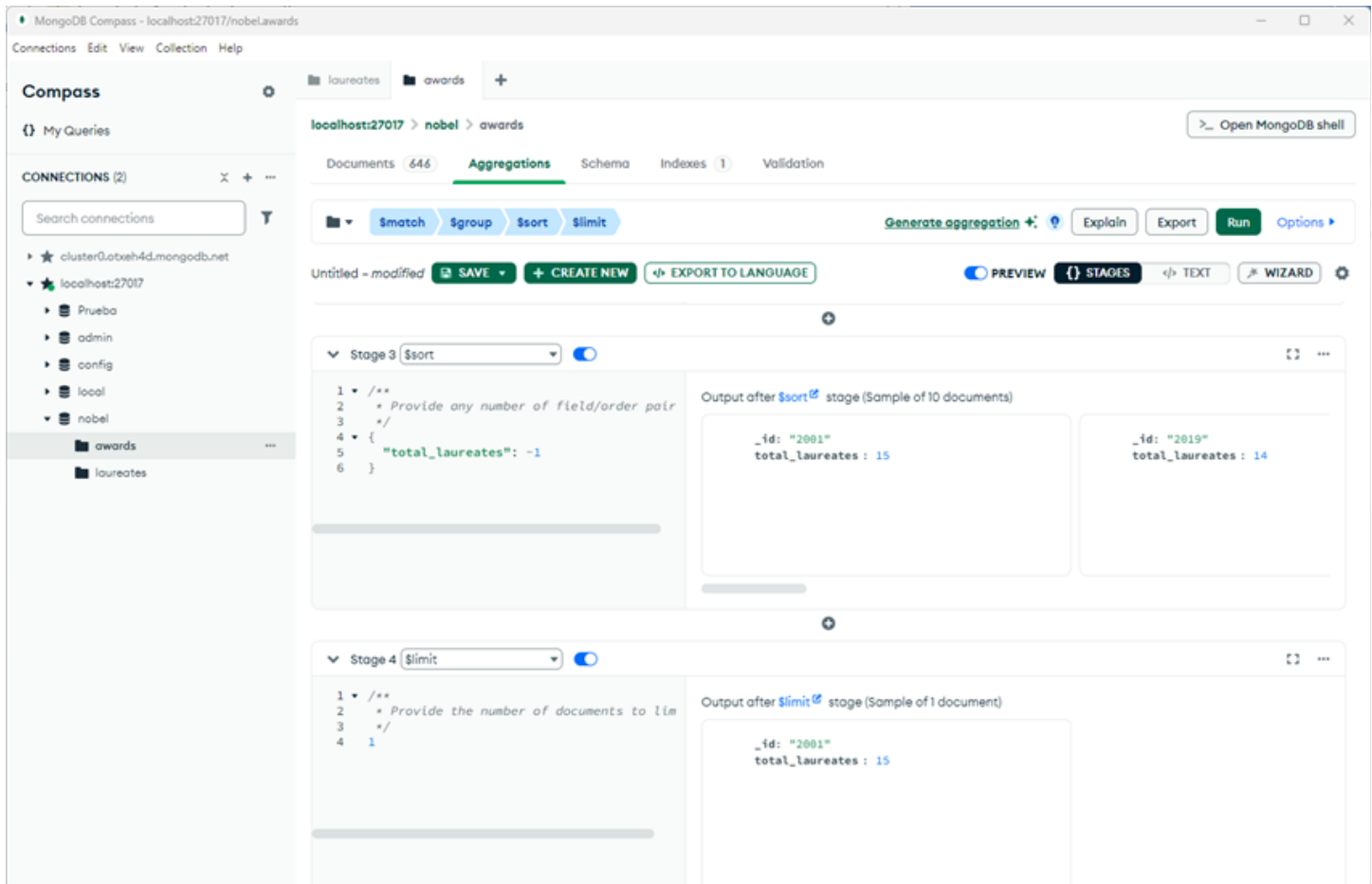
```
1 /**
2  * specifications: The fields to
3  * include or exclude.
4  */
5 {
6   "_id": "$awardYear",
7   "total_laureates": {
8     "$sum": {
9       "$size": "$laureates"
10    }
11  }
12 }
13 }
```

Output after \$group stage (Sample of 10 documents)

```
{
  "_id": "1914",
  "total_laureates": 3
}
```

Output after \$group stage (Sample of 10 documents)

```
{
  "_id": "1938",
  "total_laureates": 5
}
```



La sintaxis completa para este *pipeline* se muestra a continuación:

```
[ {
  "$match": {
    "laureates": {
      "$exists": 1
    }
  },
```

```
{
  "$group":
  {
    "_id": "$awardYear",
    "total_laureates":
    {
      "$sum": {
        "$size": "$laureates"
      }
    }
  },
  {
    "$sort":
    {
      "total_laureates": -1
    }
  },
  {
    "$limit": 1
  }
}]
```

Recuerda que el formato completo consiste en un arreglo JSON:

[etapa1, etapa2, ... , etapaN]

Antes de continuar...

Las agregaciones o *pipelines* son una de las herramientas más poderosas que ofrece MongoDB para manipular la información contenida en una colección. Como pudiste observar, nos permiten agrupar, resumir, cambiar o manipular documentos para obtener los resultados esperados.

Con esto en mente, reflexiona:

- ¿Cómo se aplicarían las agregaciones sobre la información real de una empresa?
- ¿Cómo pueden contribuir los resultados de estas agregaciones en la toma de decisiones?

Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.

I Material adicional

Devakumar K. P. (2020). *Nobel Prize Dataset* [Conjunto de datos]. Kaggle. Recuperado el 09 de junio de 2025 de: <https://www.kaggle.com/imdevskp/nobel-prize>

MongoDB Inc. (s.f.). *Aggregation Operators* [Operadores de agregación]. MongoDB. Recuperado el 09 de junio de 2025 de: <https://docs.mongodb.com/manual/reference/operator/aggregation/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 09 de junio de 2025 y su disponibilidad depende del autor.

| Glosario

Agregación: Proceso mediante el cual se agrupan, transforman y analizan datos de múltiples documentos en MongoDB para obtener resultados resumidos o compuestos.

Campo anidado: Campo dentro de un documento que contiene un objeto o estructura jerárquica. Por ejemplo, `birth.place.country.en`.

Colección: Conjunto de documentos dentro de una base de datos en MongoDB. Equivale a una tabla en bases de datos relacionales.

Etapas (\$stage): Cada bloque dentro del *pipeline* que realiza una operación específica sobre los documentos, como filtrar, agrupar, ordenar o proyectar.

Operadores de acumulación: Funciones utilizadas dentro de la etapa `$group` para operar sobre grupos de documentos. Ejemplos: `$sum`, `$avg`, `$min`, `$max`.

Operadores de agregación: Funciones que se usan dentro de las etapas del *pipeline* para transformar o manipular los datos. Varían según la etapa en la que se utilicen.

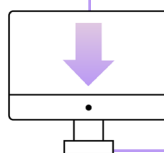
Pipeline: Estructura secuencial en la que se definen las etapas de una agregación. Cada etapa transforma los datos y los pasa como entrada a la siguiente.

Bibliografía

Bionichive. (s.f.). *BSON Specification* [Especificación BSON]. Bionichive. Recuperado el 09 de junio de 2025 de: <https://www.bionichive.com/>

MongoDB Inc. (s.f.). *Documentación de MongoDB*. MongoDB. Recuperado el 09 de junio de 2025 de: <https://docs.mongodb.com/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 09 de junio de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.